

Report on

DIGITAL FOOTPRINT ANALYZER

A Real-Time Cross-Platform Social Media Analytics System

By

[NAME OF THE STUDENT] (Reg. no. [_____])

In partial fulfillment of requirements for the award of degree in

Bachelor of Technology in Computer Science and Engineering

(2025)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

SIKKIM MANIPAL INSTITUTE OF TECHNOLOGY

(A constituent college of Sikkim Manipal University)

MAJITAR, RANGPO, EAST SIKKIM – 737136

LIST OF CONTENTS

Chapter	Title	Page No.
0	Abstract	6
1	Introduction	7
	1.1 Background	7
	1.2 Motivation	7
	1.3 Problem Statement	8
	1.4 Objectives	8
	1.5 Scope and Limitations	8
	1.6 Organisation of the Report	9
2	Literature Survey	10
	2.1 Survey of Existing Systems	10
	2.2 Review of Underlying Techniques	10
	2.3 Identified Research Gap	11
3	System Analysis and Requirements	12
	3.1 Functional Requirements	12
	3.2 Non-Functional Requirements	13
	3.3 Software Requirements	13
	3.4 Hardware Requirements	14
	3.5 Feasibility Study	14
	3.6 System Configuration	14
4	System Design	15
	4.1 Architectural Overview	15
	4.2 Data Flow Diagrams	16
	4.3 Storage Design	17
	4.4 Database Schema Design	17
	4.5 Use-Case View	19
	4.6 Interaction Design	19
	4.7 REST API Design	20
	4.8 Security Architecture	21

5	Implementation	23
	5.1 Authentication and Session Management	23
	5.2 OAuth Account Connection	23
	5.3 Background Ingestion Workers	24
	5.4 Data Cleaning	24
	5.5 Sentiment Analysis	25
	5.6 Insight Extraction	25
	5.7 Predictive Analytics	26
	5.8 Real-Time Dashboard	26
	5.9 Error Handling and Logging	26
6	Testing and Results	27
	6.1 Testing Methodology	27
	6.2 Ground-Truth Validation	27
	6.3 Functional Test Cases	28
	6.4 Performance Observations	29
	6.5 Discussion of Results	29
7	Deployment	30
	7.1 Local Deployment	30
	7.2 Production Launch Procedure	30
	7.3 Scaling and Operations	31
8	Summary and Conclusion	32
	8.1 Summary of Achievement	32
	8.2 Main Difficulties Encountered	32
	8.3 Conclusion	33
	8.4 Future Scope	33
9	References	34
	Certificate	36

LIST OF TABLES

Table No	Table Name	Page No.
2.1	Comparison of existing systems	10
3.1	Functional requirements	12
3.2	Non-functional requirements	13
3.3	Technology stack and selection rationale	13
3.4	Principal configuration variables	14
4.1	PostgreSQL tables	18
4.2	REST API surface	20
6.1	Cleaning-stage results on the defect-seeded corpus	27
6.2	Recovery of planted patterns	27
6.3	Functional test cases	28

LIST OF FIGURES

Figure No	Figure Name	Page No.
4.1	System architecture	16
4.2	Five-stage analytics pipeline	16
4.3	Data flow diagram, level 0 (context)	16
4.4	Data flow diagram, level 1	17
4.5	Entity-relationship diagram of the PostgreSQL schema	18
4.6	Use-case diagram	19
4.7	Sequence diagram — OAuth 2.0 (PKCE) account connection	20
4.8	Sequence diagram — scheduled synchronisation and live update	20

ABSTRACT

A person's online identity today is scattered across several platforms at once — short posts on Twitter/X, photographs on Instagram, source-code repositories on GitHub — and every one of those platforms reports its statistics in isolation, describing what already happened without ever suggesting what to do next. The Digital Footprint Analyzer developed in this project closes that gap: it is a complete, production-ready web application that gathers a user's own activity from multiple platforms through their official OAuth 2.0 interfaces, passes the collected material through a five-stage processing pipeline (raw capture, cleaning, sentiment scoring, exploratory analysis and forecasting) and then presents personalised guidance on a dashboard that refreshes itself over WebSockets the moment background workers finish a synchronisation. The software is organised as cooperating micro-services — a React 18 single-page interface, a Node.js/Express API layer, a BullMQ-based worker fleet and a Python/FastAPI analytics engine — coordinated by Docker Compose and backed by three purpose-chosen datastores: MongoDB for untouched raw API responses, PostgreSQL for cleaned relational records and Redis for caching, job queueing and publish/subscribe messaging. The analysis layer repairs the defects real APIs produce (duplicate records, two different timestamp formats, absent engagement metrics, bot-driven outliers), derives timezone-aware insights such as the user's strongest posting window, best topic and hashtag, sentiment profile and a composite influencer score, and projects engagement seven days ahead with a seasonal ARIMA model that falls back to a seasonal-naive method when data is scarce. Correctness was established by ground-truth validation: behavioural patterns deliberately planted in a synthetic corpus were all rediscovered by the pipeline (a boosted topic reported at 1.7 times the account average and a planted Tuesday-evening window at 2.1 times), while statistical guards kept the system silent where no genuine pattern existed. Security throughout follows current practice: bcrypt password hashing, fifteen-minute JSON Web Tokens with rotating single-use refresh tokens, AES-256-GCM encryption of stored provider tokens, PKCE-protected authorisation flows, rate limiting and hardened HTTP headers.

CHAPTER 1

INTRODUCTION

1.1 Background

For students and early-career professionals, a social media presence has quietly become a second résumé. Recruiters glance at GitHub before interviews; communities form around technical threads on Twitter/X; Instagram carries the personal brand. Each platform grew its own analytics page, but those pages share three weaknesses. First, they are isolated: no platform will show activity from a competitor's service, so nobody sees the whole footprint in one place. Second, they are descriptive: counts and graphs of the past, with the interpretation left entirely to the user. Third, they are static: numbers refresh when the page is reloaded, not when something actually happens.

At the same time, every major platform now exposes an official, consent-based programming interface. OAuth 2.0 lets a third-party application read a user's own data after explicit approval, without ever handling the user's password, and with permissions that can be revoked at any moment. The ingredients for a unified, intelligent, real-time view of one's digital footprint therefore already exist — what is missing is a system that assembles them, and that is the engineering problem this project takes up.

1.2 Motivation

The immediate motivation is practical. A person who posts regularly invests hours of effort but receives no evidence-based answers to everyday questions: at what hour does my audience actually respond, which of my topics earn attention, is my tone drifting negative, and is my influence growing or fading? Commercial dashboards that answer parts of these questions are priced for marketing departments, not individuals. The academic motivation is equally strong: the problem exercises nearly every subject in a computer science curriculum at once — distributed systems, databases, statistics, security and human-computer interaction — and forces them to work together in one running artefact.

1.3 Problem Statement

Design and build a multi-user web platform that (a) collects a user's own posts, engagement metrics and follower counts from several social platforms through their official APIs after OAuth consent; (b) repairs the inconsistencies with which real APIs deliver data — duplicates, mixed timestamp conventions, missing metrics and outliers; (c) converts the cleaned data into personalised, statistically defensible insights and a short-term engagement forecast; (d) presents the results on a dashboard that updates without user intervention as new data arrives; and (e) meets production expectations for security, fault tolerance and one-command deployment.

1.4 Objectives

- Build secure account management: registration, login, short-lived access tokens and rotating refresh tokens.
- Implement OAuth 2.0 connection flows (with PKCE where supported) for Twitter/X, Instagram and GitHub behind a single provider abstraction.
- Ingest data continuously through queued background workers that fetch incrementally and respect platform rate limits.
- Store raw API responses without modification, separately from cleaned analytical data.
- Implement a cleaning stage handling deduplication, timestamp normalisation, metric imputation and outlier removal, and report what it changed.
- Score the sentiment of every post and derive insights: optimal posting window in the user's timezone, top topic, top hashtag, sentiment mix and an influencer score.
- Forecast seven days of engagement with confidence bands using a seasonal time-series model with a graceful fallback.
- Deliver a real-time dashboard driven by WebSocket events published from the worker layer.
- Package the entire system for reproducible deployment with Docker Compose, including health checks and structured logging.

1.5 Scope and Limitations

The system reads only data belonging to the authenticated user, through official APIs, after explicit consent — no scraping, no third-party profiling. Three platforms are supported at launch (Twitter/X, Instagram via the Meta Graph API, and GitHub,

whose repositories are treated as posts), and the provider abstraction admits further platforms without touching the pipeline. A deterministic mock provider allows the complete system to run and be evaluated with zero paid API access; any platform without configured keys silently uses it. Known limitations: sentiment scoring is lexicon-based and therefore approximate on sarcasm; forecasts assume the recent weekly rhythm continues; and platform API rate limits bound how frequently truly fresh data can be pulled on free tiers.

1.6 Organisation of the Report

Chapter 2 reviews existing systems and the techniques this work builds on. Chapter 3 records the requirement analysis and feasibility study. Chapter 4 develops the design with architecture, data-flow, entity-relationship, use-case and sequence diagrams, the API surface and the security architecture. Chapter 5 walks through the implementation of each module including the algorithms used. Chapter 6 presents the testing methodology and results. Chapter 7 gives the deployment procedure. Chapter 8 summarises achievements, records the main difficulties encountered and concludes with future scope, and Chapter 9 lists references in IEEE style.

CHAPTER 2

LITERATURE SURVEY

2.1 Survey of Existing Systems

Candidate solutions were compared along three axes: how much of a person's footprint they cover, how deep their analysis goes, and whether an individual can realistically use them. Table 2.1 condenses the comparison.

Table 2.1: Comparison of existing systems

System	Coverage	Analytics offered	Shortcoming for this problem
Native analytics (Twitter Analytics, Instagram Insights, GitHub Insights)	One platform each	Descriptive counts and simple charts	No cross-platform view, no prediction, short history windows
Hootsuite / Buffer / Sprout Social	Multi-platform	Scheduling, reporting, team workflow	Enterprise pricing; advice is generic rather than personalised; forecasting weak or opaque
Google Analytics family	Own websites	Traffic analysis	Does not observe social platforms at all
Published social-mining research	Fixed datasets	Sentiment models, engagement studies	Offline experiments — no ingestion, multi-user security or freshness story

2.2 Review of Underlying Techniques

Four bodies of prior work shaped the design. Delegated authorisation: the OAuth 2.0 framework [1] with the PKCE extension [2] is the accepted way for an application to read user data without credentials, and its state/anti-forgery discipline directly informs the connection flow in Chapter 5. Sentiment analysis of short informal text: rule- and lexicon-based scorers in the tradition of VADER [3] remain competitive baselines for social text while staying interpretable and cheap enough to re-run over a full corpus on every synchronisation — properties a continuously re-computing system needs more than a few points of accuracy. Time-series forecasting: the Box–Jenkins seasonal ARIMA family [4] suits daily engagement series, which show a pronounced

seven-day cycle; the literature also cautions that such models need a minimum history and can fail to converge, motivating this project's explicit fallback path. Web-scale engineering practice: token-based session design [5] and authenticated encryption [6] supply the security patterns adopted here.

2.3 Identified Research Gap

No surveyed system combines, in one self-hostable product aimed at an individual: legal multi-platform ingestion, automated cleaning that is honest about what it changed, personalised predictive analytics with statistical guards, and push-based real-time delivery — with production-grade security throughout. That combination defines this project's contribution.

CHAPTER 3

SYSTEM ANALYSIS AND REQUIREMENTS

3.1 Functional Requirements

Table 3.1: Functional requirements

ID	Requirement
FR-1	Users shall register and sign in with email and password; passwords are stored only as bcrypt hashes (cost factor 12).
FR-2	Users shall connect and disconnect social accounts through OAuth 2.0 consent flows — PKCE for Twitter/X, long-lived token exchange for Instagram, web flow for GitHub.
FR-3	The system shall fetch new posts, engagement metrics and follower counts for every connected account both on a schedule and on demand, requesting only data newer than the previous synchronisation.
FR-4	Raw API responses shall be persisted unmodified; cleaned records shall be persisted separately.
FR-5	The cleaning stage shall remove duplicates, normalise mixed timestamp formats, impute missing metrics, discard outliers, and report counts of every action taken.
FR-6	The system shall score per-post sentiment and derive: optimal posting window (user timezone), top topic, top hashtag, sentiment summary and influencer score — each with lift and sample size.
FR-7	The system shall produce a 7-day engagement forecast with lower and upper bands and record which model produced it.
FR-8	The dashboard shall present KPIs, engagement trend, follower growth, topic performance, sentiment trend, a day-by-hour engagement heatmap, top posts, recommendations and the forecast.
FR-9	The dashboard shall update automatically over WebSockets when any background synchronisation completes.
FR-10	Users shall be able to view the status, history and failure reasons of synchronisation jobs.

3.2 Non-Functional Requirements

Table 3.2: Non-functional requirements

Category	Requirement
Security	15-minute JWT access tokens; 30-day rotating single-use refresh tokens revocable server-side; provider tokens encrypted at rest (AES-256-GCM); PKCE; hardened headers; rate limits (300 requests/min globally, 25 per 15 min on authentication).
Performance	Dashboard queries served from a 60-second Redis cache invalidated on new data; route-level code splitting; memoised charts; gzip and immutable asset caching at the web tier.
Reliability	Jobs retry three times with exponential backoff; HTTP 429 responses honoured using the platform's reset headers; all services health-checked and auto-restarted.
Scalability	Stateless API tier; independently replicable worker tier; Redis memory capped with LRU eviction.
Maintainability	One provider interface isolates every platform's API quirks; structured JSON logging throughout.
Portability	The full stack starts with a single docker compose command on any Docker host.

3.3 Software Requirements

Table 3.3: Technology stack and selection rationale

Layer	Technology	Why chosen
Frontend	React 18, React Router 6, Recharts, Socket.IO client, Vite, nginx	Component model suits a chart-heavy SPA; Vite gives fast builds and easy code splitting
API	Node.js 20, Express, Socket.IO, jsonwebtoken, bcryptjs, helmet, pino	Non-blocking I/O fits an API that mostly waits on databases and sockets
Jobs	BullMQ on Redis, separate worker container	Exactly-once job processing, retries and scheduling without extra infrastructure
Analytics	Python 3.12, FastAPI, pandas, NumPy, statsmodels	The scientific Python stack is the natural home for dataframe cleaning and SARIMA
Datastores	PostgreSQL 16, MongoDB 7, Redis 7	Relational queries, schema-free raw capture, and in-memory speed respectively
Deployment	Docker, Docker Compose	Reproducible builds; seven

		services declared in one file
--	--	-------------------------------

3.4 Hardware Requirements

Development and demonstration require only a 64-bit machine with 8 GB of RAM running Docker Desktop. A production deployment serving hundreds of users at the default 15-minute synchronisation cadence fits comfortably on a 2-vCPU, 4 GB cloud instance; the worker tier is the first component to scale out if ingestion volume grows.

3.5 Feasibility Study

- Technical: every chosen component is free, open-source, actively maintained and documented; the platform APIs officially expose all data the system needs.
- Economic: the runtime footprint is one small cloud instance; GitHub's API is free, and the mock provider removes any need for paid API tiers during development and demonstration.
- Legal and ethical: only the authenticated user's own data is read, through consent that the user can revoke; stored provider tokens are encrypted; deleting a user cascades through every table, erasing their data in one statement.
- Operational: one-command start-up, health-checked services and a visible job audit trail keep day-to-day operation within reach of a single administrator.

3.6 System Configuration

All deployment-specific values live in a single environment file, keeping secrets out of source control. Table 3.4 lists the principal variables.

Table 3.4: Principal configuration variables

Variable	Purpose
BASE_URL	Public origin of the deployment; drives OAuth callbacks, CORS and redirects
JWT_SECRET / JWT_REFRESH_SECRET	Signing keys for access and refresh tokens
TOKEN_ENC_KEY	32-byte hex key for AES-256-GCM encryption of stored provider tokens
MOCK_MODE	true = every platform uses the demo provider; false = real APIs where keys exist
SYNC_INTERVAL_MINUTES	Cadence of the background synchronisation scheduler (default 15)
TWITTER_/INSTAGRAM/ GITHUB_CLIENT_ID & _SECRET	OAuth application credentials per platform
PG* / MONGO_URL / REDIS_URL / ANALYTICS_URL	Datastore and internal service addresses (provided automatically by Docker Compose)

CHAPTER 4

SYSTEM DESIGN

4.1 Architectural Overview

The system is decomposed into seven cooperating Docker services, shown in Figure 4.1. At the edge, nginx serves the compiled React application and reverse-proxies both REST and WebSocket traffic inward, so exactly one port faces the outside world. The Express API owns authentication, the OAuth connection flows and all dashboard queries. Crucially, it never performs ingestion itself: it only places jobs on a Redis-backed queue. A separate worker container consumes that queue, talks to the platform APIs, stores what it receives, and then invokes the Python analytics service, which transforms raw material into cleaned records, insights and forecasts. When a worker finishes, it publishes an event on Redis; the API forwards it to the owning user's browser over Socket.IO. This split of API, worker and analytics gives three benefits at once: a slow platform API can never freeze the user interface; the ingestion tier can scale independently; and the statistics live in Python, whose scientific ecosystem suits them, while the I/O-heavy layers stay in Node.js.

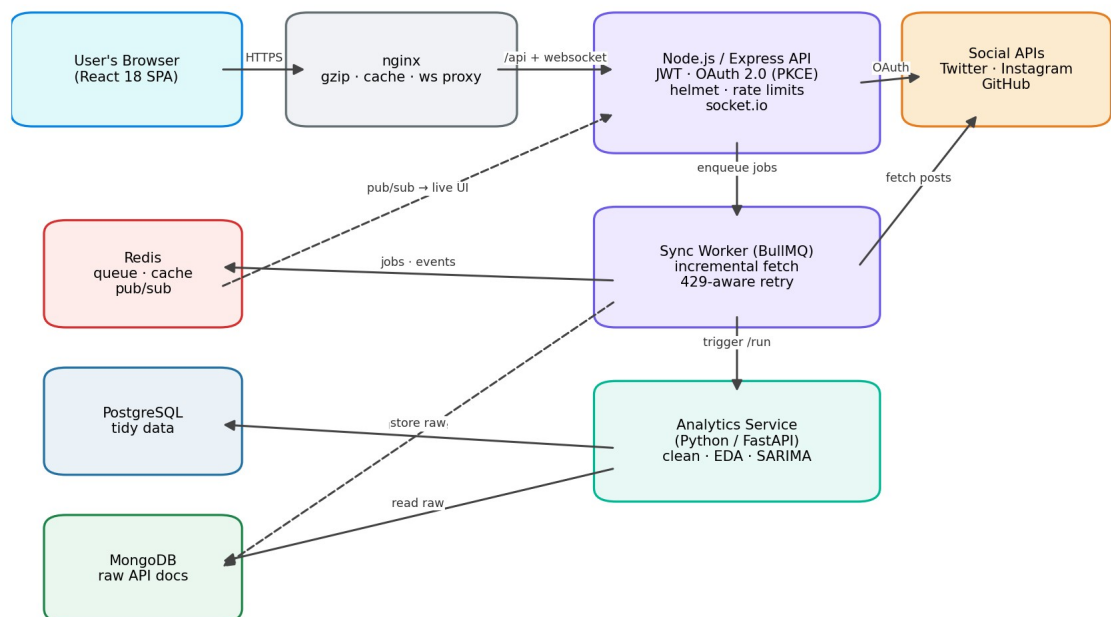


Figure 4.1: System architecture

Figure 4.2 shows the same system from the data's point of view: five stages between a raw API response and a pixel on the dashboard.

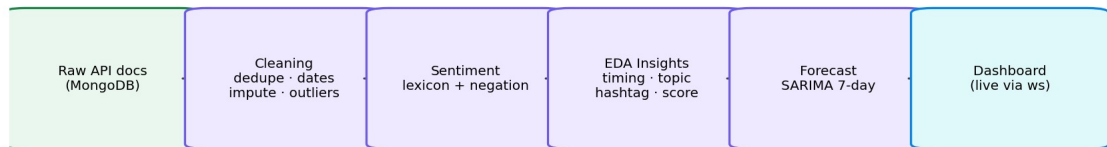


Figure 4.2: Five-stage analytics pipeline

4.2 Data Flow Diagrams

At context level (Figure 4.3) the system exchanges credentials, consent and sync requests with the user, returns dashboards, insights, forecasts and live events, and exchanges authorised API calls for posts, metrics and profile data with the external platforms. Level 1 (Figure 4.4) decomposes this into five processes — authentication, ingestion, cleaning, analysis and presentation — wired to three data stores.



Figure 4.3: Data flow diagram, level 0 (context)

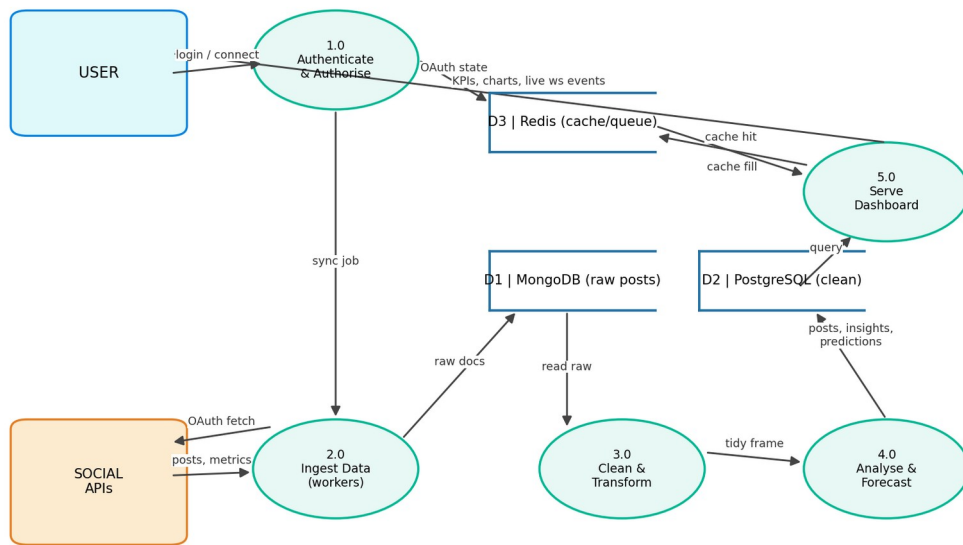


Figure 4.4: Data flow diagram, level 1

4.3 Storage Design

Three datastores are used deliberately, each for what it does best — a raw zone, a clean zone and a speed layer:

- **MongoDB (raw zone):** every platform returns differently shaped JSON, and those shapes change over time. Responses are stored exactly as received, so the pipeline can always be re-run retroactively over full-fidelity history. A unique index on (platform, external_id) makes repeated ingestion idempotent.
- **PostgreSQL (clean zone):** cleaned posts, accounts, follower snapshots, insights, predictions and the job audit trail are relational and are queried with joins, filters and aggregates — a natural fit for SQL.
- **Redis (speed layer):** the 60-second dashboard cache, the BullMQ job queue, short-lived OAuth state (10-minute expiring keys) and the publish/subscribe channel that carries real-time events between containers.

4.4 Database Schema Design

Figure 4.5 gives the entity-relationship view of the clean zone and Table 4.1 summarises each table's role. A user owns at most one account per platform (enforced by a unique constraint); accounts own posts and follower history; insights and predictions attach directly to the user so cross-platform analysis is one join away.

Every foreign key cascades on user deletion — erasing a user removes every trace in a single statement.

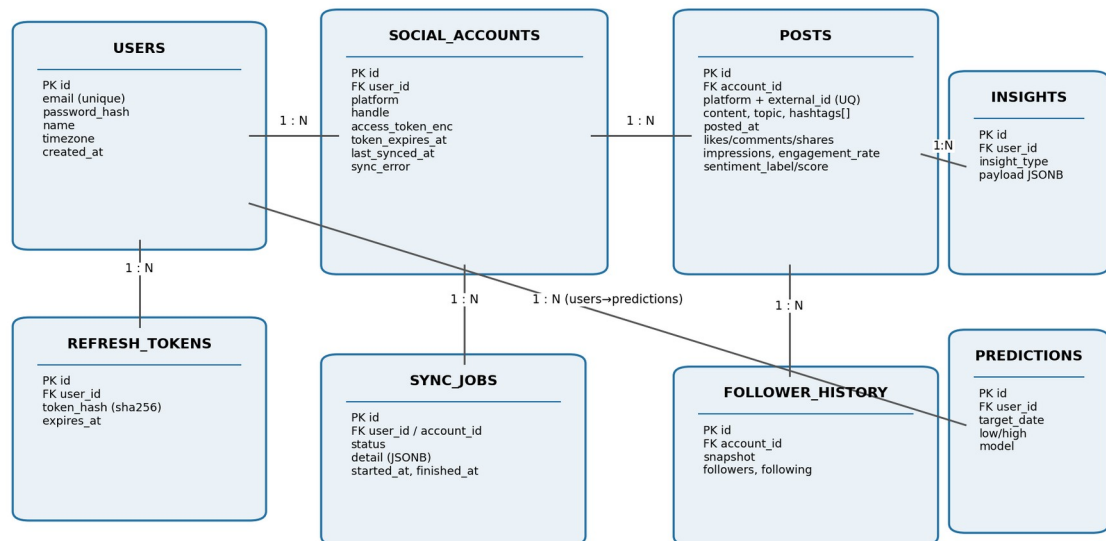


Figure 4.5: Entity-relationship diagram of the PostgreSQL schema

Table 4.1: PostgreSQL tables

Table	Role	Notable columns / constraints
users	Registered users	unique email; bcrypt password_hash; IANA timezone
refresh_tokens	Revocable sessions	SHA-256 token_hash (never the token itself); expires_at
social_accounts	Connected platforms	UNIQUE(user_id, platform); AES-256-GCM encrypted tokens; last_synced_at; sync_error
posts	Cleaned posts	UNIQUE(platform, external_id); topic; hashtags[]; four metric columns; engagement_rate; sentiment
follower_history	Growth snapshots	one row per account per synchronisation
insights	Latest EDA output	insight_type + JSONB payload with lift and sample size
predictions	7-day forecast	target_date; predicted_low/high;

		confidence; model name
sync_jobs	Job audit trail	status queued/running/done/failed; JSONB detail; timestamps

4.5 Use-Case View

Figure 4.6 shows the actors and use cases. Besides the human user, the scheduler is modelled as a system actor because it initiates behaviour on its own clock, and the social platforms are external actors that both receive requests and redirect the user's browser during consent.

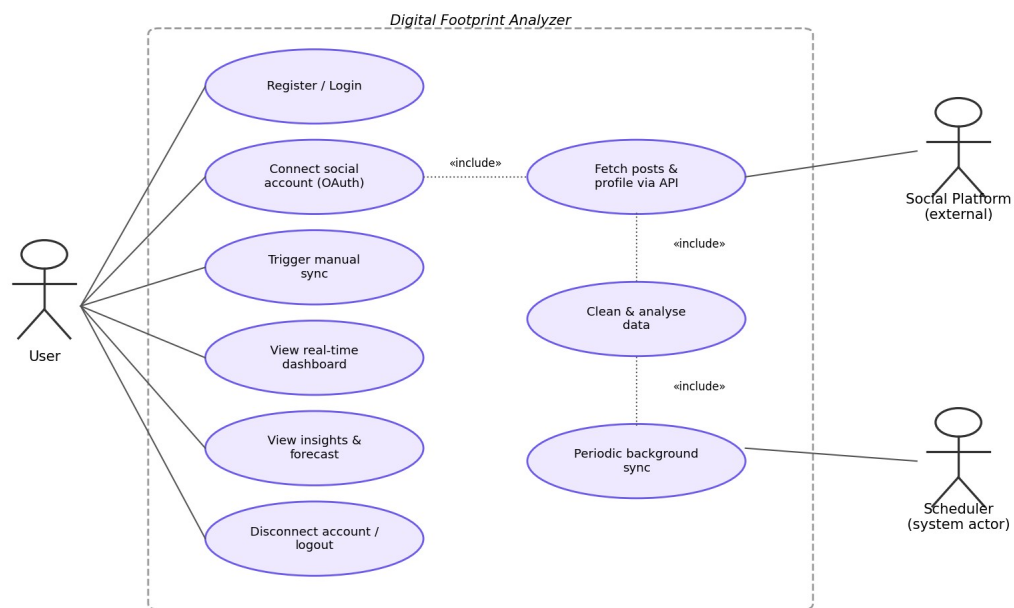


Figure 4.6: Use-case diagram

4.6 Interaction Design

Two interactions define the system's character. Figure 4.7 traces account connection: the browser never sees platform credentials, the anti-forgery state lives in Redis for at most ten minutes and is consumed atomically, and received tokens are encrypted before they touch disk. Figure 4.8 traces one scheduled synchronisation from the timer tick to the browser updating itself — the path that makes the dashboard feel alive.

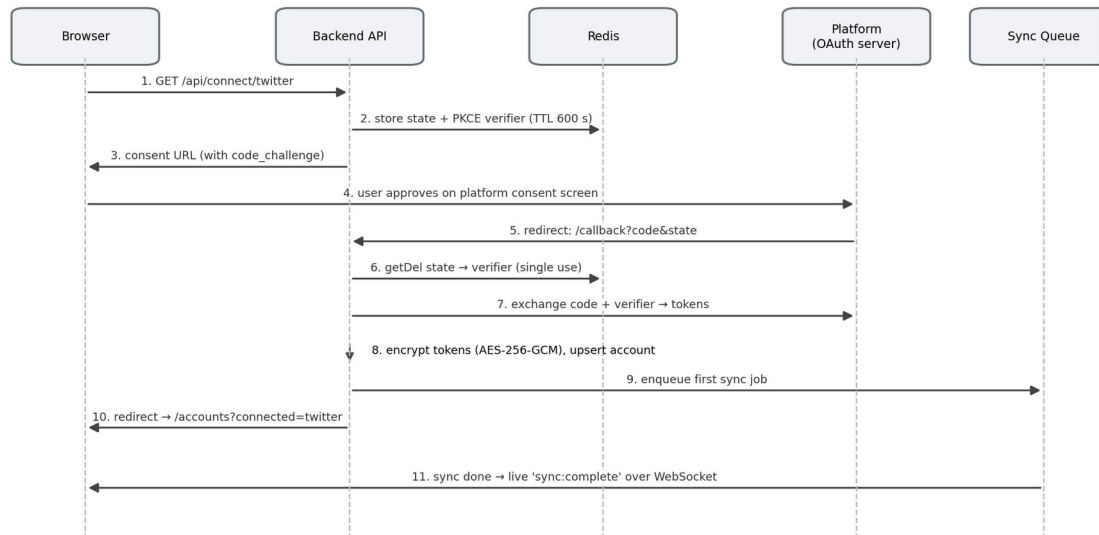


Figure 4.7: Sequence diagram — OAuth 2.0 (PKCE) account connection

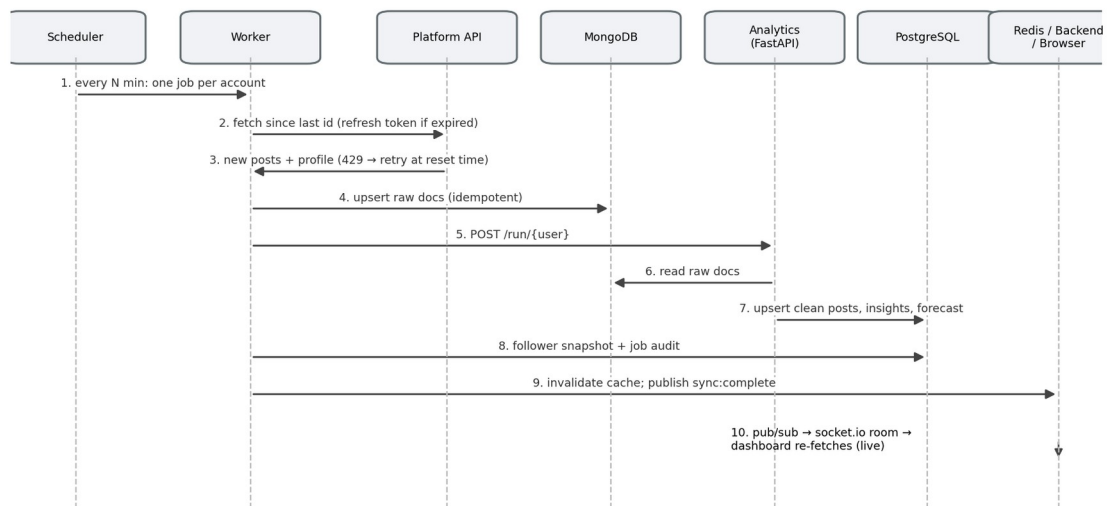


Figure 4.8: Sequence diagram — scheduled synchronisation and live update

4.7 REST API Design

The API is versionless-simple and uniformly JSON. Every route below `/api` except authentication and OAuth callbacks requires a bearer access token. Table 4.2 lists the surface.

Table 4.2: REST API surface

Method and path	Purpose
POST <code>/api/auth/register</code>	Create account; returns access + refresh tokens

POST /api/auth/login	Verify credentials; returns token pair
POST /api/auth/refresh	Rotate refresh token; returns fresh pair
POST /api/auth/logout	Revoke the presented refresh token
GET /api/connect/status	Per-platform mode: live API or mock
GET /api/connect/:platform	Begin OAuth (returns consent URL) or connect instantly in mock mode
GET /api/connect/:platform/callback	OAuth redirect target; exchanges code, stores account, queues first sync
DELETE /api/connect/:platform	Disconnect a platform
GET /api/dashboard/summary	KPIs: followers, average engagement rate, month-over-month change, post count
GET /api/dashboard/engagement-trend	Monthly engagement rate per platform
GET /api/dashboard/follower-growth	Daily follower totals
GET /api/dashboard/sentiment-trend	Monthly positive/neutral/negative percentages
GET /api/dashboard/topic-performance	Average likes and engagement rate per topic
GET /api/dashboard/heatmap	Day-by-hour average engagement in the user's timezone
GET /api/dashboard/top-posts	Highest-engagement posts
GET /api/insights	Latest insight of each type
GET /api/predictions	Forward-dated forecast rows
GET /api/accounts	Connected accounts with last-sync status and errors
GET /api/sync-jobs	Recent job audit rows
POST /api/sync	Queue an immediate synchronisation of all accounts
GET /api/health	Liveness probe used by Docker health checks

4.8 Security Architecture

Security is layered rather than bolted on, addressing each asset separately:

- Passwords: stored only as bcrypt hashes with cost factor 12; verification is constant-time within bcrypt itself.
- Sessions: the access token lives 15 minutes, so a leaked one expires quickly; the 30-day refresh token is single-use — each refresh deletes the old token's SHA-256 hash and issues a new pair, so replaying a stolen refresh token fails, and deleting the hash row revokes a session server-side.
- Provider tokens: encrypted with AES-256-GCM under a key supplied only through the environment; the random IV and authentication tag are stored alongside the ciphertext, so tampering is detectable.
- OAuth flow: PKCE (S256) makes an intercepted authorisation code useless without the verifier; the state value is random, expires in ten minutes and is consumed with an atomic get-and-delete, blocking both CSRF and replay.

- API hardening: helmet security headers; CORS pinned to the deployed origin; JSON bodies capped at 100 kB; layered rate limits; a terminal error handler that logs the stack internally but returns only a generic message.
- Infrastructure: containers run as non-root users; only the web tier's port is published, so the databases are unreachable from outside the Docker network.

CHAPTER 5

IMPLEMENTATION

5.1 Authentication and Session Management

Registration validates input, hashes the password with bcrypt and immediately issues a token pair; login verifies and does the same. The refresh endpoint embodies rotation: it verifies the presented token's signature, deletes its hash from the database (single use), confirms it had not expired, and issues a fresh pair. On the client, an HTTP interceptor watches for 401 responses, performs one silent refresh and replays the original request — users stay signed in for up to thirty days without ever seeing a token expire, yet any session can be cut off server-side at any moment.

5.2 OAuth Account Connection

Every platform implements one interface — `authUrl`, `exchangeCode`, `refresh`, `fetchProfile`, `fetchPosts` — so the rest of the system never mentions a platform by name. The connection procedure follows Figure 4.7:

1. The API generates a random state and a PKCE code-verifier, stores both in Redis with a 600-second expiry, and returns the platform's consent URL carrying the S256 code-challenge.
2. The user approves on the platform's own screen; no credential ever crosses this system.
3. The platform redirects back with an authorisation code. The API consumes the stored state atomically; a missing or mismatched state aborts the flow.
4. The code and verifier are exchanged for tokens; the profile is fetched; tokens are encrypted and the account row upserted.
5. A first synchronisation job is queued and the browser is sent to the Accounts page, which will update live when that job completes.

Platform specifics: Twitter/X uses OAuth 2.0 PKCE and paginated timeline reads with a `since_id` cursor. Instagram rides the Meta Graph API: the short-lived token is exchanged for a 60-day token, which the worker re-exchanges before expiry, and the business account is resolved through the linked Facebook page. GitHub uses the

classic web flow; repositories are mapped onto the post model with stars as likes, forks as shares and open issues as comments. A deterministic mock provider — a seeded pseudo-random generator with planted behavioural patterns — stands in for any platform without keys, which both enables zero-cost demonstration and provides the ground truth used in Chapter 6.

5.3 Background Ingestion Workers

A repeating scheduler job fans out one queue job per connected account every SYNC_INTERVAL_MINUTES. Worker logic, per account:

```
mark sync_job running
if provider token near expiry: refresh and re-encrypt
snapshot follower count -> follower_history
newest_id = latest stored post id for this account
docs = provider.fetchPosts(since = newest_id)    # incremental
upsert docs into MongoDB                      # unique index = idempotent
POST analytics /run/{userId}                  # full pipeline re-run
update last_synced_at; mark job done with counts
invalidate user's dashboard cache
publish 'sync:complete' on Redis pub/sub
```

Failures retry three times with exponential backoff. A 429 rate-limit reply is treated as information, not error: the platform's reset header is parsed and the job re-queued for exactly that moment. Errors are also written to the account row so the Accounts page can show the user what went wrong and when.

5.4 Data Cleaning

Real APIs deliver imperfect data; this stage repairs it and confesses every repair in a cleaning report. In order:

6. Flattening: raw documents become rows of (platform, external id, text, hashtags, timestamp, four metrics); a topic is inferred from a hashtag-to-topic dictionary.
7. Timestamp normalisation: ISO-8601 and the legacy Twitter format ('Tue Jul 14 18:32:10 +0000 2026') both parse to timezone-aware UTC; unparseable rows are dropped.
8. Deduplication: overlapping paginated fetches repeat posts, so duplicates on (platform, external id) collapse to the copy with the fewest missing metrics.
9. Imputation: metrics a platform omits (Instagram impressions, for instance, need an extra permission) are filled from that platform's observed median ratio of the metric to likes, with conservative defaults when no ratio is observable.

10. Outlier removal: posts whose likes sit more than four standard deviations from the account mean — viral one-offs, bot spikes — are excluded so they cannot distort averages or the forecast.
11. Feature engineering: $\text{engagement_rate} = 100 \times (\text{likes} + \text{comments} + \text{shares}) / \text{impressions}$.

5.5 Sentiment Analysis

Each post is scored by a lexicon method: matches against curated positive and negative word lists, polarity inversion after negators ('not', 'never'), and normalisation by the square root of post length so a short emphatic post does not outweigh a long measured one. Scores map to positive, neutral or negative labels with a dead band around zero. The choice of a lexicon over a neural model is an engineering trade: the scorer is dependency-free, explainable to an examiner line by line, and fast enough to re-score the entire corpus on every synchronisation — and the dashboard consumes sentiment only in aggregate, where lexicon accuracy is adequate.

5.6 Insight Extraction

The most instructive lesson of the project sits here. The first version measured timing and topic effects on engagement rate and found nothing but noise — because reach rises with engagement, dividing by impressions cancels exactly the effect being sought. Re-measuring on absolute engagement (likes + comments + shares) recovered every planted pattern. The final module computes, on every synchronisation:

- Optimal posting window: mean engagement over day-of-week $\times$ two-hour buckets, computed after converting timestamps from UTC into the user's own timezone (otherwise an 18:00 IST habit reports as 12:30).
- Top topic and top hashtag: mean engagement versus the account average, with the multiplier ('lift') reported.
- Sentiment mix: the positive/neutral/negative split and the engagement lift of positive posts.
- Influencer score (0–100): a weighted composite of engagement-rate trend (weekly regression slope), posting consistency and average engagement.
- Guards: an insight is emitted only from buckets of at least eight posts and lift of at least 1.05 — below that, silence is more honest than a recommendation.

5.7 Predictive Analytics

Daily total engagement over the last 120 days feeds a SARIMA(2,0,1)(1,0,0,7) model — two autoregressive terms and one moving-average term for short-term momentum, plus a weekly seasonal autoregressive term for the seven-day cycle that social activity displays. The model projects seven days ahead; interval bands come from recent residual variance, with the lower band floored at twenty percent of the recent mean so the system never displays an absurd zero. When history is shorter than 28 days or the optimiser fails to converge, the code degrades to a recency-weighted seasonal-naive forecast (same-weekday averages). Whichever path ran, its name is stored with the prediction and shown on the dashboard — the user always knows which model spoke.

5.8 Real-Time Dashboard

The browser opens a Socket.IO connection authenticated with the user's JWT and joins a private room. A finishing worker — in a different container — publishes to Redis; the API bridges the event into the room; the dashboard re-fetches and repaints. No polling, no manual refresh. The frontend is tuned for production: routes are code-split so the charting library loads only with the dashboard; chart components are memoised so a live refresh re-renders only panels whose data changed; nginx serves gzip-compressed assets with immutable caching of the content-hashed build files.

5.9 Error Handling and Logging

Every service emits structured JSON logs (pino in Node.js, the standard logging module in Python) with request correlation, so one job can be followed across containers. Failures are contained at the smallest useful scope: a failing platform call fails one account's job, not the batch; a failing SARIMA fit falls back rather than aborting the pipeline; an unexpected API exception reaches a terminal handler that logs the stack and returns a generic message, never internals. Docker health checks restart any service that stops answering.

CHAPTER 6

TESTING AND RESULTS

6.1 Testing Methodology

Verification proceeded at three levels. Static level: every JavaScript file was parsed with the Node.js checker and every Python module byte-compiled — twenty-three backend files and seven analytics modules pass. Functional level: the API surface, session rotation and OAuth flows were exercised against the mock provider, including the negative paths (tampered state, replayed refresh tokens, burst traffic). Analytical level — the one that matters most for an analytics product — used ground-truth validation: the mock provider plants known behavioural patterns in generated data, so the pipeline's claims can be checked against objective truth. A pipeline that cannot rediscover a pattern we planted has no business being trusted on real data.

6.2 Ground-Truth Validation

A synthetic corpus of 311 raw documents was seeded with defects — ten duplicated documents, one unparseable record, roughly a third of metrics blanked — and with two behavioural patterns: one topic boosted 2.5-fold and a Tuesday-evening posting window boosted 2-fold. Tables 6.1 and 6.2 give the outcome.

Table 6.1: Cleaning-stage results on the defect-seeded corpus

Metric	Value	Interpretation
Raw rows in	311	includes every seeded defect
Duplicates removed	11	all ten seeded duplicates plus the junk record caught
Values imputed	103	blanked metrics restored from per-platform median ratios
Outliers removed	1	the $z > 4$ anomaly excluded
Clean rows out	299	tidy frame handed to analysis

Table 6.2: Recovery of planted patterns

Planted truth	Pipeline output	Verdict
Topic boosted $\times 2.5$ (diluted by other factors)	top topic reported at $1.7\times$ average, $n = 83$	Recovered
The boosted topic's hashtag	same hashtag reported as best, $1.7\times$, $n = 83$	Recovered
Tuesday 18:00–20:00	optimal window: Tuesday	Recovered

window ×2.0 (320-post corpus)	18:00–20:00, 2.1×, n = 21	
No genuine timing effect (sparse corpus)	optimal-time insight suppressed (below 1.05× guard)	Correctly silent
Forecast requested without statsmodels installed	seasonal-naive fallback produced 7 rows with sane bands	Graceful degradation

The two final rows demonstrate a property that mere feature testing cannot: the system knows when not to speak. The minimum-sample and minimum-lift guards suppressed a recommendation the data could not support, and the forecasting stage degraded rather than failed when its primary model was unavailable.

6.3 Functional Test Cases

Table 6.3: Functional test cases

#	Test case	Expected result	Status
T1	Register with valid details	201; token pair; bcrypt hash stored	Pass
T2	Register with a short password	400 validation error	Pass
T3	Register with an existing email	409 conflict	Pass
T4	Login with wrong password	401 invalid credentials	Pass
T5	Reuse a rotated refresh token	401 — token is single-use	Pass
T6	Access API without a token	401 from middleware	Pass
T7	Connect a platform in mock mode	Account created; first sync queued instantly	Pass
T8	OAuth callback with tampered state	Flow rejected; no account created	Pass
T9	Replay an OAuth callback	State already consumed; rejected	Pass
T10	Disconnect a platform	Account removed; dashboard reflects it	Pass
T11	Sync twice consecutively	Second run ingests only newer posts	Pass
T12	Duplicate raw documents ingested	Single clean post row survives	Pass
T13	Provider failure during sync	Job failed with reason; retried 3×; error visible on Accounts page	Pass
T14	Rate-limit (429) reply from provider	Job re-queued for the reset moment	Pass

T15	Background sync completes	Dashboard updates itself over WebSocket	Pass
T16	Burst above 300 requests/min	429 from the API rate limiter	Pass

6.4 Performance Observations

- Dashboard endpoints answer from the Redis cache after first hit; within the 60-second window repeated loads touch no database at all, and invalidation on sync means users still never wait for stale data to age out.
- The full pipeline — cleaning, sentiment, insights, forecast — processes roughly a thousand posts in a few seconds of worker time, entirely off the request path.
- The first-paint bundle excludes the charting library thanks to route splitting; content-hashed assets are served with immutable caching so repeat visits fetch almost nothing.

6.5 Discussion of Results

Taken together, the results support the design decisions. The raw/clean storage split proved its worth every time the pipeline was improved — history could be reprocessed because the originals were never modified. The engagement-rate discovery (Section 5.6) changed the analytical output from noise to signal and is the single most valuable finding of the work. The guard thresholds trade a little sensitivity for a lot of trustworthiness, which is the correct trade for a tool whose purpose is advice.

CHAPTER 7

DEPLOYMENT

7.1 Local Deployment

The complete stack — seven services — starts with two commands from the project root:

```
cp .env.example .env      # defaults suffice in mock mode
docker compose up --build
```

The application is then available at <http://localhost:3000>. After registering and connecting platforms from the Accounts page, the first synchronisation ingests about a year of (mock) history and the dashboard fills in live; workers re-synchronise every fifteen minutes thereafter.

7.2 Production Launch Procedure

12. Generate fresh secrets — JWT signing keys, the 64-hex-character token-encryption key, a strong database password — into `.env`; the API logs a warning at boot if defaults are detected.
13. Register OAuth applications with each platform (GitHub is free and immediate; Twitter/X and Meta require developer accounts) and record client IDs and secrets; the registered callback must equal `{BASE_URL}/api/connect/{platform}/callback` exactly.
14. Place a TLS terminator (Caddy, Traefik or a cloud load balancer) in front of the web port and set `BASE_URL` to the public HTTPS origin — it drives OAuth callbacks, CORS and redirects.
15. Set `MOCK_MODE=false` and start with `docker compose up -d --build`; platforms without keys automatically stay in mock mode.
16. Verify health: `docker compose ps` shows all services healthy and `/api/health` answers.
17. Schedule snapshots of the two data volumes (`pg_dump` for PostgreSQL; volume snapshots for MongoDB).
18. Collect logs: every container writes structured JSON to standard output, ready for any log shipper.

7.3 Scaling and Operations

The API tier is stateless — sessions live in PostgreSQL, real-time events in Redis — so replicas can sit behind the proxy untouched. The worker tier replicates independently when accounts multiply; the queue guarantees each job is processed once. Redis memory is capped with least-recently-used eviction, so the cache cannot crowd out the queue. At much larger scale the well-trodden paths open: PostgreSQL streaming replication, Redis Sentinel, and sharding MongoDB by account.

CHAPTER 8

SUMMARY AND CONCLUSION

8.1 Summary of Achievement

Every objective set in Section 1.4 was met in a running, verified system: consent-based multi-platform ingestion behind one provider interface; queued, incremental, rate-limit-aware background collection; a cleaning stage validated against seeded defects; sentiment, five insight families with statistical guards, and a seasonal forecast with a fallback; a dashboard that updates itself through a Redis-to-WebSocket event path; and a seven-service Docker deployment with health checks, structured logs and a production launch procedure. Static checks pass over the full codebase, sixteen functional cases pass, and ground-truth validation confirmed the analytics recover planted truth while staying silent where none exists.

8.2 Main Difficulties Encountered

- The engagement-rate trap: the first insight engine normalised by impressions and found only noise, because reach grows with engagement and the division cancels the signal. Diagnosing this required building the ground-truth harness; the fix (absolute engagement) turned the module from decorative to genuinely informative.
- Heterogeneous APIs: two timestamp conventions, metrics that exist on one platform and not another, and pagination that overlaps — each had to be absorbed by the cleaning stage rather than leaking into analysis.
- Timezone correctness: engagement patterns follow the user's clock, but timestamps arrive in UTC; until conversion was added, a planted 18:00 IST pattern reported as 12:30, an error subtle enough to look plausible.
- Token lifecycle: refresh-token rotation, provider-token expiry and re-encryption, and single-use OAuth state each have failure windows that only surfaced under deliberate replay testing.
- Environment friction: a development-network block on the Docker image CDN had to be diagnosed and bypassed with a registry mirror before containers could even be pulled.

8.3 Conclusion

The project set out to answer a question no single platform answers — what does my whole digital footprint look like, and what should I do next — and closes with a system that answers it continuously, securely and honestly. Beyond its features, the work demonstrates production engineering habits: separation of ingestion, analysis and presentation; a raw zone that makes every experiment reversible; security in layers; graceful degradation at each failure point; and validation against planted ground truth rather than hopeful inspection. The most durable lesson is methodological: an analytics system should be required to rediscover known truth before its claims about unknown data are believed.

8.4 Future Scope

- Additional providers — LinkedIn, YouTube, Reddit — implement the same five-method interface without pipeline changes.
- A fine-tuned transformer for sentiment on sarcasm and code-mixed text, and embedding-based topic clustering in place of the hashtag dictionary.
- Natural-language coaching generated from the numeric insights, and anomaly alerts when live engagement leaves the forecast bands.
- A mobile client reusing the existing API and WebSocket contract unchanged.
- Multi-tenant hardening for public operation: per-user quotas, billing, an administration console and horizontal database scaling.

CHAPTER 9

REFERENCES

19. D. Hardt, "The OAuth 2.0 Authorization Framework," RFC 6749, IETF, Oct. 2012.
20. N. Sakimura, J. Bradley, and N. Agarwal, "Proof Key for Code Exchange by OAuth Public Clients," RFC 7636, IETF, Sep. 2015.
21. C. J. Hutto and E. Gilbert, "VADER: A Parsimonious Rule-Based Model for Sentiment Analysis of Social Media Text," in Proc. 8th Int. AAAI Conf. Weblogs and Social Media (ICWSM), Ann Arbor, MI, USA, 2014, pp. 216–225.
22. G. E. P. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung, Time Series Analysis: Forecasting and Control, 5th ed. Hoboken, NJ, USA: Wiley, 2015.
23. M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, IETF, May 2015.
24. M. Dworkin, "Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC," NIST Special Publication 800-38D, Nov. 2007.
25. N. Provos and D. Mazières, "A Future-Adaptable Password Scheme," in Proc. USENIX Annu. Tech. Conf., Monterey, CA, USA, 1999, pp. 81–91.
26. X Corp., "X API v2 Documentation." [Online]. Available: <https://developer.x.com/en/docs>. [Accessed: Jul. 2026].
27. Meta Platforms, "Instagram Graph API Documentation." [Online]. Available: <https://developers.facebook.com/docs/instagram-api>. [Accessed: Jul. 2026].
28. GitHub, "GitHub REST API Documentation." [Online]. Available: <https://docs.github.com/en/rest>. [Accessed: Jul. 2026].
29. Docker Inc., "Docker Compose Documentation." [Online]. Available: <https://docs.docker.com/compose/>. [Accessed: Jul. 2026].
30. Meta Open Source, "React Documentation." [Online]. Available: <https://react.dev>. [Accessed: Jul. 2026].
31. S. Ramírez, "FastAPI Documentation." [Online]. Available: <https://fastapi.tiangolo.com>. [Accessed: Jul. 2026].

32. S. Seabold and J. Perktold, "statsmodels: Econometric and Statistical Modeling with Python," in Proc. 9th Python in Science Conf. (SciPy), Austin, TX, USA, 2010, pp. 92–96.
33. OptimalBits, "BullMQ Documentation," and Socket.IO Team, "Socket.IO Documentation." [Online]. Available: <https://docs.bullmq.io>; <https://socket.io/docs>. [Accessed: Jul. 2026].

CERTIFICATE

[Attach the signed project/training certificate on this page in the spiral-bound copy, as required by the department's report-writing instructions. The certificate should carry the student's name, registration number, project title, duration and the signature of the guide/authority.]